6-1 · 실시간 데이터와 Kafka · 실습 2.5H

Kafka 핵심 구조와 첫 실행

토픽·파티션·오프셋을 잡고, 브로커를 띄워 첫 메시지

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

2.5시간 뒤에는

① 브로커·토픽·파티션·오프셋을 우체국 비유로 설명한다

② Docker 로 KRaft 단일 브로커 Kafka 를 기동한다

③ CLI 로 토픽을 만들고 메시지를 보내고 받는다

④ 읽어도 사라지지 않는 것과 「같은 키 = 같은 파티션」을 눈으로 확인한다

오늘의 kafka 컨테이너와 hello-stream 토픽은 개념용 — 6-2 부터 compose 로 정식 구성합니다

우체국으로 한 번에 — 편지는 읽어도 사라지지 않는다

Producer	토픽(Topic)	파티션(Partition)	Consumer
편지를 부치는 사람	이벤트 종류별 사서함 벽	개별 사서함 — 많을수록 동시에	자기 사서함을 확인하러 오는 사람

브로커 = 우체국 건물 · 오프셋 = 파티션 안 일련번호 · KRaft = ZooKeeper 없이 스스로 메타데이터 관리

큐와 다른 점 하나 — Kafka 의 편지는 보존 기간 동안 그대로 남아, 여러 부서(컨슈머 그룹)가 각자 자기 속도로 같은 편지를 읽는다

순서는 파티션 안에서만 보장된다

1 **같은 키 → 언제나 같은 파티션**
키의 해시로 정해진다 — 한 파티션 안에서는 넣은 순서대로 읽힌다

2 **파티션이 여러 개면 전체 순서는 없다**
파티션끼리는 독립 — 「주문 ID별 순서」가 필요하면 주문 ID 를 키로

3 **키를 안 주면 한 칸에 몰아 담는다**
16KB 가 차면 다음 칸으로 — 몇 줄이면 모두 한 파티션(실측)

전체 순서가 꼭 필요하면

파티션 1개

= 병렬 없음

> **대부분의 업무는 「같은 대상 안에서의 순서」만 필요 — 키를 잘 고르는 것이 설계 · Spark 파티션처럼 병렬 처리의 단위**

실습 흐름

일곱 단계 — ~/data-eng/week6 에서 docker exec 로

터미널 두 개를 씁니다 — 하나는 Consumer 를 켜 두고, 다른 하나에서 Producer 로 입력합니다

1. 기동	2. 토픽	3·4. 송수신	5·6. 키·오프셋	7. 미니 과제
apache/kafka 3.7.0	hello-stream 파티션 3	재시작해도 남아 있나	같은 키 같은 파티션	파티션 1개 --group

1·2단계

핵심 명령 — 기동 · 확인 · 토픽

기동	docker run -d --name kafka -p 9092:9092 apache/kafka:3.7.0
떴는지	docker ps --filter name=kafka
준비됐는지	docker logs kafka | grep -i "kafka server started" — 몇 초 뒤
토픽 만들기	kafka-topics.sh --create --topic hello-stream --partitions 3
토픽 상세	kafka-topics.sh --describe --topic hello-stream

CLI 는 컨테이너 안 /opt/kafka/bin — 앞에 docker exec kafka /opt/kafka/bin/ 과 --bootstrap-server localhost:9092 를 붙인다

3~6단계

핵심 명령 — 보내기 · 읽기 · 키 · 오프셋

Consumer	kafka-console-consumer.sh --topic hello-stream --from-beginning
Producer	kafka-console-producer.sh --topic hello-stream
키 함께 보내기	--property parse.key=true --property key.separator=:
키·파티션 출력	--property print.key=true --property print.partition=true
오프셋	kafka-get-offsets.sh --topic hello-stream

console producer 는 한 줄이 메시지 하나 · A:첫 번째 / A:두 번째 / D:다른 키 / A:세 번째 순으로 입력해 본다 · 콜론 없는 줄은 Producer 를 멈춘다

3~6단계

단계마다 확인 포인트 — 오늘의 핵심 관찰은 4단계

1. **3. 터미널 2 입력이 터미널 1 에 곧바로**
   Producer 한 줄 = 메시지 하나 · 한글도 그대로

2. **4. Consumer 를 끄고 --from-beginning 으로 다시**
   Ctrl+C 에 Processed a total of 3 messages — 다시 켜면 전부 다시 나온다

3. **5. 키 A 는 Partition:1, D 는 Partition:0**
   A 셋은 입력 순서 그대로 — 키 없이 보낸 것은 null, 한 칸에 모여 있다

4. **6. hello-stream:0:5 · 1:3 · 2:0 (실측)**
   더 보내고 다시 실행하면 N 이 늘어난다 — 오프셋은 일련번호

--from-beginning 을 빼면

지금부터만 —

새로 보낸 것만

2단계 — 일부러 실패시키기

replication-factor 2 로 먼저 만들어 보기

「복제본 없음」이 무슨 뜻인지 실패로 확인합니다 — 2단계 토픽을 만든 뒤 hello-rf2 로

factor 2 로	만들어지지 않는다	이유	운영에서는
--replication-factor 2 hello-rf2	only 1 broker(s) are registered	브로커 1대 — 복제본 둘 곳 없음	브로커 3대 복제 3

확인 포인트

이 2.5시간이 끝날 때 확인할 것

1. docker ps 의 STATUS 가 Up · PORTS 에 0.0.0.0:9092→9092/tcp
2. describe 에 PartitionCount: 3 · Partition 0/1/2 · Leader 1
3. 터미널 2 에 입력한 문장이 터미널 1 에 곧바로 나타난다
4. Consumer 를 --from-beginning 으로 다시 켜면 이전 메시지가 전부
5. 키 A 의 세 메시지가 Partition:1 에 입력 순서대로, D 는 Partition:0
6. kafka-get-offsets.sh 의 hello-stream:0:N 이 보낼수록 늘어난다

6-1 · 실습 2.5H 마무리

다음은 정리·회고 (0.5H)

용어를 우체국 비유로 한 장에 정리하고, 6-2 클러스터 구성을 위한 준비를 확인합니다

- ✓ 용어 8개 · CLI 명령 7개 요약
- ✓ 자주 겪는 오류 다섯 가지
- ✓ docker rm -fv kafka · 이미지는 남기기
- ✓ Total Memory 7.7GiB 안팎 확인

가져갈 것

kafka-get-offsets.sh 가 찍은 파티션별 오프셋